Esta parte del examen se aprueba respondiendo correctamente al menos el 75 % de las preguntas. Es necesario aprobar esta parte para acceder a la corrección de la segunda parte. Responder las preguntas en los espacios indicados.

**Pregunta 1:** La siguiente definición ¿utiliza recursión explícita? Justificar.

```
esHoja :: AB a -> Bool
esHoja Nil = False
esHoja (Bin i r d) = esNil i && esNil d
  where esNil Nil = True
        esNil _ = False
```

**Respuesta:** NO porque no se define ninguna función que se llame así misma (no hay llamado recursivo).

**Pregunta 2:** Indicar el tipo del esquema de recursión estructural foldABNV para el siguiente tipo:

```
data ABNV a = Hoja a | Uni a (ABNV a) | Bi (ABNV a) a (ABNV a)
```

**Respuesta:** foldABNV :: (a → b) → (a → b → b) → (b → a → b → b) → ABNV a → b

**Pregunta 3:** ¿Qué tipo de recursión utiliza la siguiente definición? (Indicar la más específica).

```
listasQueSuman :: Int -> [[Int]]
listasQueSuman 0 = [[]]
listasQueSuman n | n > 0 = [x : xs | x <- [1..n], xs <- listasQueSuman (n-x)]
```

a. Estructural

b. Primitiva

c. Global

d. Iterativa/a la cola

**Respuesta:** c

**Pregunta 4:** Indicar cuál de las siguientes divisiones en casos puede justificarse por un Lema de Generación.

a. Hay 2 casos:

- altura i ≥ altura d
- altura i < altura d

(Con i, d :: AB a).

b. La lista xs puede ser:

- []
- [x]
- x:y:ys

(Con x, y :: a, ys :: [a]).

c. El árbol binario t puede ser:

- Nil
- Bin i r d

(Con r :: a, i y d :: AB a).

**Respuesta:** c

**Pregunta 5:** Definir el predicado unario P que se podría usar para demostrar la siguiente propiedad por inducción estructural:

∀ xs::[a] . ∀ ys::[a] . length (entrelazar xs ys) = length xs + length ys

**Respuesta:** P(xs) = ∀ ys::[a]. length (entrelazar xs ys) = length xs + length ys

Para demostrar la propiedad, se demuestra que ∀xs::[a] vale P(xs).

**Pregunta 6:** Si se quisiera demostrar que vale $\neg P \vee Q \vdash Q \vee (P \Rightarrow Q)$ por deducción natural, ¿cuál sería el error al aplicar el siguiente paso?

$$\frac{\neg P \vee Q \vdash Q}{\neg P \vee Q \vdash Q \vee (P \Rightarrow Q)}\ (\vee_{i1})$$

en el contexto.

**Respuesta:** El error es que pierdo información necesaria para poder demostrarlo, ya que con $\neg P \vee Q$ no tengo forma de probar que siempre vale Q.

**Pregunta 7:** Indicar a qué término reduce **en un paso** la siguiente expresión, y mediante qué regla(s):

if isZero(zero) then Pred(Succ(zero)) else zero

**Respuesta:** reduce a if true then Pred(Succ(zero)) else zero, por reglas E-IF y E-IsZero.

**Pregunta 8:** Sea la extensión del Cálculo Lambda con listas vista en la guía:

$$M ::= \ldots \mid [\,]_\tau \mid M :: M \mid \text{case } M \text{ of } \{[\,] \rightsquigarrow M \mid h :: t \rightsquigarrow M\} \mid \text{foldr } M \text{ base} \rightsquigarrow M; \text{rec}(h, r) \rightsquigarrow M$$

Indicar el error en la siguiente regla de cómputo para **foldr**:

$$\frac{}{\text{foldr } V_1 :: V_2 \text{ base} \rightsquigarrow M; \text{rec}(h, r) \rightsquigarrow N \to N}\ (\text{E-FOLDR-CONS})$$

**Respuesta:** El término N ~~tiene~~ puede tener apariciones libres de las variables h y r, por lo que al reducir el foldr $V_1 :: V_2$ es necesario ligarlas con la cabeza y el resultado de la recursión sobre la cola. La regla debería ser:

$$\text{foldr } V_1 :: V_2 \text{ base} \rightsquigarrow M; \text{rec}(h, r) \rightsquigarrow N \longrightarrow *$$

$$* \; N\{h := V_1, r := \text{foldr } V_2 \text{ base} \rightsquigarrow M; \text{rec}(h, r) \rightsquigarrow N\}$$

CamScanner